

Remove N^{th} Node From End of List

Walmart

Oracle

Goldman Sachs

Pattern: Fixed Gap Family

Problem Definition & Core Idea

Given the head of a singly-linked list, remove the n^{th} node from the end of the list and return its head.

Core Algorithmic Idea: The problem belongs to the **Fixed Gap Family** of pointer patterns. The primary goal is to establish and maintain a constant visual gap between a `slow` pointer and a `fast` pointer so that we can identify the correct target node relative to the end of the list in a highly efficient manner.

Walkthrough Example

Consider a linked list: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ with $n = 2$.

To determine which node corresponds to the n^{th} node from the end, we can use the position formula:

$$\text{Position from start (1-indexed)} = (\text{len} - n + 1) = (5 - 2 + 1) = 4^{\text{th}} \text{ node}$$

$$\text{Index from start (0-indexed)} = (\text{len} - n) = (5 - 2) = 3^{\text{rd}} \text{ index}$$

Hence, the 4^{th} node (value 4) needs to be removed from the list, transforming it into $1 \rightarrow 2 \rightarrow 3 \rightarrow 5$.

Approach 1: Brute Force (Two-Pass)

The brute-force method breaks down the problem into two sequential passes over the linked list:

- **Step 1 (First Pass):** Traverse the list from the beginning to compute the total length (len).

- **Step 2 (Edge Case Handling):** If $len == n$, the node to remove is the absolute first node (the head). We handle this explicitly by storing $head \rightarrow next$, deleting head, and returning the new head pointer.
- **Step 3 (Second Pass):** To safely delete the node at index $(len - n)$, we must stop our traversal exactly one node before it—at index $(len - n - 1)$. This allows us to modify the pointer chains and re-establish a secure connection.

Time Complexity	$O(len) + O(len) = O(2 \cdot len) \Rightarrow O(n)$ — Requires two separate iterations.
Space Complexity	$O(1)$ — Operates entirely in-place with no extra heap allocation.

Brute Force Code Structure

```
// S1: Find length of linked list
int len = 0;
ListNode* temp = head;
while(temp) {
    len++;
    temp = temp->next;
}

// S2: Handle edge case (removing the head node)
if (len == n) {
    ListNode* newhead = head->next;
    delete(head);
    return newhead;
}

// S3: Navigate to the (len - n - 1)-th position
int pos = len - n - 1;
ListNode* curr = head;
while (pos--) {
    curr = curr->next;
}

// Re-establish pointer connection and delete target node
ListNode* tbd = curr->next;
curr->next = curr->next->next;
delete(tbd);
return head;
```

Approach 2: Optimal Approach (One-Pass / Fast & Slow Gap)

Instead of scanning the list twice, we can use the **Fast & Slow Pointer Gap Technique** to eliminate the node in a single traversal pass.

The Core Logic: If we create a fixed window gap of n nodes between two pointers, when the fast pointer reaches the termination bound (NULL), the slow pointer will be located precisely at the n^{th} node from the end.

However, to smoothly excise a node from a singly linked list, we must be positioned at the node **immediately prior** to the target node. Therefore, we deliberately maintain a gap of $n + 1$ nodes.

Why a Dummy Node is Crucial: Creating a mock dummy node at the start of the list guarantees that the slow pointer has a valid landing block even if the target node to delete is the original head itself. It cleanses the implementation of awkward edge-case checks.

- **S1:** Initialize a dummy node with `val = 0` and `next = head`. Set both `slow` and `fast` to the dummy node.
- **S2:** Move the fast pointer forward by $n + 1$ steps. For $n = 2$, fast advances 3 steps, establishing a robust pointer separation.
- **S3:** Advance both pointers forward simultaneously one step at a time until fast turns into NULL.
- **S4:** The slow pointer is now sitting right before the node targeted for elimination. Bridge the connection: `slow->next = slow->next->next` and clean up the unlinked memory.

Time Complexity	$O(n)$ — Solved gracefully in a single consolidated pass.
Space Complexity	$O(1)$ — Only requires two extra tracker pointers.

Visual Trace of Optimal Gap Execution ($n = 2$)

1. Initial State (Setting up Dummy & Pointers)



2. After Advancing Fast Pointer by $n + 1 = 3$ Steps (Gap Established)



3. Final Loop Position (When Fast Reaches NULL)



Official C++ Source Implementation (Unchanged)

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *   int val;
 *   ListNode *next;
 *   ListNode() : val(0), next(nullptr) {}
 *   ListNode(int x) : val(x), next(nullptr) {}
 *   ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode* dumdum=new ListNode(0);
        dumdum->next=head;

        ListNode* slow=dumdum;
        ListNode* fast=dumdum;

        for(int i=0;i<=n;i++){
            fast=fast->next;        //last se n+1 node jaane ke liye , n+1 ka gap maintain
            //kareng
        }

        while(fast){
            slow=slow->next;
            fast=fast->next;
        }

        ListNode* tbd=slow->next;
        slow->next=slow->next->next;
        delete(tbd);
        return dumdum->next;
    }
};
```